

Synchronous Serial: SPI

Add the clock wire back. Four signals, four modes, simultaneous two-way exchange, and the two settings that make the difference between data and zeros

Textbook: Dean, *Embedded Systems Fundamentals*, 2nd ed., Chapter 8, printed pages 249–257.

Lab manual: Lab 08 (SPI with the I3G4250D gyroscope). Your own Lab 08 report and project plan.

1 What we are actually after this week

Last week's UART had no clock line, so both ends had to guess the timing from a start bit and a preagreed baud rate. This week we spend one more wire and see what it buys.

The question: **what changes when the transmitter sends its clock along with the data?**

Quite a lot, and not all of it good. The plan:

1. The **four signals** and the topology, including how one master addresses several slaves without any addressing in the protocol.
2. **Simultaneous exchange**: the property that makes SPI genuinely different from UART, and that confuses everyone at first.
3. **CPOL and CPHA**: the four SPI modes, and why picking the wrong one gives you zeros rather than an error.
4. The peripheral: registers, the **baud rate divisor**, FIFOs, and status flags.
5. Transmitting and receiving, including a subtle **compiler** problem that transmits a phantom byte.
6. **The register-access convention** that every SPI sensor uses, worked against your own gyroscope.

Item 3 is the exam question and the single most common cause of a dead SPI link. Item 5 is a bug you would never guess. Item 6 is Lab 08, and I will use your own configuration throughout.

2 Deep dive 1: The four signals

SPI

Serial Peripheral Interface: a synchronous serial protocol with one **master** and one or more **slaves**. Typically the MCU is the master and the peripheral devices are slaves.

Common slave devices: ADCs, accelerometers, LCD controllers, magnetometers, flash memory, and gyroscopes.

Recall the terminology note from Week 14: these names date from the 1980s and persist in every register bit name, so you must recognise them even where newer alternatives are preferred.

Signal	Meaning	Direction and role
SCK	SPI clock	Master → slaves. The master alone generates it
MOSI	Master Out, Slave In	Master → slaves. Data going out
MISO	Master In, Slave Out	Slaves → master. Data coming back
NSS/CS	Slave select	Master → one line per slave

If communication only needs to go one way, you can drop MISO or MOSI accordingly.

How SPI does addressing without addressing

Recall Week 14's problem 6: with several receivers, the transmitter must indicate which one it means, either via **separate select signals** or **inside the message**.

SPI chooses separate select signals. There is no address field anywhere in an SPI transaction. The master asserts exactly one slave-select line, and the selected device responds while every other device ignores the bus entirely.

The consequence, and it is the main structural weakness of SPI: **n slaves need n select pins**. Three sensors is seven wires. Ten sensors is fourteen. Recall the pin-count argument from Week 14 that motivated serial communication in the first place, and notice SPI partially reintroduces it. That is precisely the gap I²C fills next week.

3 Deep dive 2: Simultaneous exchange

Here is the property that makes SPI different, and it genuinely surprises people.

SPI is a circular shift register

The master and the slave each hold a shift register, and the two are wired into a **loop** via MOSI and MISO. **Each clock pulse exchanges one bit between them.**

So for an 8-bit transfer, eight clock pulses **simultaneously** send one byte out and receive one byte in. **There is no such thing as transmit-only or receive-only.** Every SPI transaction is an exchange.

Contrast with UART, where transmission and reception are independent and have separate hardware.

The sequence: the master asserts the slave's select line, drives SCK, and puts data on MOSI; the slave samples MOSI while the master samples MISO. After the last bit shifts in, the receiver sets a status flag and may raise an interrupt. Then the select line can be released, though sometimes it is held.

Beware the trap

This is why reading a sensor register takes two bytes' worth of clocking.

To read one register you must send a command byte, and *something* comes back during those eight clocks, but it is meaningless because the slave has not yet been told what you want. Then you must clock **eight more** times to shift the answer out, and during those clocks you are transmitting something, usually a dummy byte such as 0x00 or 0xFF.

So "read one byte" is really "exchange two bytes and keep the second". If you see SPI code sending dummy bytes and wonder why, this is why: **the clock only runs while the master transmits**, so you must transmit in order to receive.

4 Deep dive 3: CPOL and CPHA, the four modes

Two settings define the relationship between clock and data. Because each is binary, **there are four versions of SPI**, and different peripherals use different ones.

The two settings

CPOL (clock polarity) determines whether the clock is **active-high** (CPOL= 0, idles low) or **active-low** (CPOL= 1, idles high).

CPHA (clock phase) determines when valid data appears and is sampled.

CPHA= 0: master and slave both present valid data **as soon as slave select is asserted**. The **first** clock edge (inactive→active) marks the middle of the bit time, so both sample then. The next edge advances the shift registers. *Slave select may remain active between transfers.*

CPHA= 1: data is not presented until the inactive→active edge and is sampled on the **active→inactive** edge, the **second** edge. *Slave select must go inactive between transfers.*

Mode	CPOL	CPHA	Clock idles / samples on
0	0	0	Low / first edge (rising)
1	0	1	Low / second edge (falling)
2	1	0	High / first edge (falling)
3	1	1	High / second edge (rising)

Beware the trap

This is the single most common reason an SPI link returns nothing but zeros or garbage.

And here is what makes it nasty: **there is no error flag for a mode mismatch**. Recall from Week 14 that UART at least gives you a framing error when the timing is wrong. SPI has no framing, no parity, and no acknowledgment. The slave samples at the wrong instants, shifts out whatever it shifts out, and everyone proceeds cheerfully.

So the symptom of wrong CPOL/CPHA is identical to the symptom of a disconnected wire, an unpowered sensor, or a chip select that never went low. **Your only real diagnostic is a logic analyser**, which is why Week 14 pushed you to get one.

The mode is not negotiable and not discoverable. **Read it out of the slave's datasheet**, every time.

In the lab

Your own project plan lists this exact failure, and it is worth quoting back at you because you already worked it out:

Under "Common Mistakes", for WHO_AM_I returning 0x00 from the gyroscope, you list (a) CPOL or CPHA wrong, which must be **CPOL High, CPHA 2nd Edge**, (b) PE3 chip select not pulled low before the transaction, and (c) the gyroscope not powered on because CTRL_REG1 was never written.

Those three are exactly the failure modes of this section: wrong mode, wrong selection, wrong device state. All three produce the same symptom.

CPOL High with CPHA 2nd Edge is **SPI mode 3**, which is what the I3G4250D and the L3GD20 both require, and it matches your Lab 08 configuration table.

5 Deep dive 4: The SPI peripheral

Book board vs your board

Book's F091RC: **two** identical SPI peripherals, SPI1 and SPI2.

Your F303VCT6: more of them, and **SPI1 is already wired to the on-board gyroscope**, which is why Lab 08 uses it without your choosing.

The pin assignments differ, so do not copy the book's.

Book: SCK on PB3, MISO on PB4, MOSI on PB5, NSS on PA15.

Yours (from your Lab 08): PA5 SCK, PA6 MISO, PA7 MOSI, with PE3 as chip select driven as a plain GPIO output.

At the core is the shift register. Around it are two **FIFO** buffers, TXFIFO and RXFIFO, each holding up to four bytes, which ease the timing requirements on your software by giving it more slack before data is lost. Both are accessed through SPIx_DR: writing transmits, reading receives.

5.1 SPIx_CR1

Field	Meaning
MSTR	Master (1) or slave (0)
LSBFIRST	LSB first (1) or MSB first (0)
CPOL	Active-high (0) or active-low (1) clock
CPHA	Sample on first edge (0) or second edge (1)
BR	Baud rate divisor, master mode only
SPE	SPI enable. Set to 1 <i>only after configuring everything else</i>

Beware the trap

SPE deserves its own warning. Setting it not only starts the peripheral, it **disables certain configuration changes**. So configure first, enable last, exactly as with the timer's CEN in Week 12.

Trying to change CPOL after enabling the peripheral is a classic silent no-op: your code looks right, the register does not change, and the link stays broken.

5.2 Baud rate

The master divides the bus clock by 2^{BR+1} , giving factors of 2, 4, 8, 16, up to 256:

$$\text{Baud rate} = \frac{f_{\text{BusClock}}}{2^{BR+1}}$$

Book's worked example: BR= 3 gives $48 \text{ MHz} / 2^4 = 48/16 = 3 \text{ MHz}$, or 333 ns per bit.

In the lab

Your Lab 08's arithmetic, from your own report: the system clock runs at 48 MHz from the PLL with an HSE source and a multiplication factor of 6. APB2 also runs at 48 MHz and feeds SPI1. With a prescaler of 8, the SPI clock is $48/8 = 6 \text{ MHz}$, comfortably inside the sensor's 10 MHz limit.

Note the prescaler of 8 corresponds to BR = 2, since $2^{2+1} = 8$. CubeMX shows you the

human number and writes the encoded one.

The manual recommends a prescaler of 8 or 16 for this sensor, with the constraint stated as ≤ 10 MHz. **That ceiling comes from the slave, not the master**, and this is the general rule: an SPI link runs at the speed of its slowest participant, and exceeding it produces, once again, silent garbage.

5.3 SPIx_CR2 and the status register

CR2 holds DS, the data size, which defaults to 8 bits after reset and can be set from 4 to 16 bits. It also holds TXEIE, RXNEIE, and ERRIE for interrupts, DMA enables, and FRXTH, the receive FIFO threshold.

Status flags in SPIx_SR:

Flag	Meaning
TXE	Transmit buffer empty
RXNE	Receive FIFO not empty, at least one item available
FTLVL, FRLVL	FIFO fill level: 00 empty, 01 quarter, 10 half, 11 more than half
BSY	Busy communicating, or transmit buffer not empty
MODF	Mode fault: multiple masters trying to drive SCK and MOSI

Beware the trap

FRXTH is a real trap and the book flags it. Data longer than 8 bits occupies *two* FIFO entries, so RXNE must be told what counts as "something available".

Set FRXTH to **0 for data longer than eight bits** and to **1 otherwise**. Get it wrong with 16-bit data and RXNE fires after half a value has arrived, so you read a byte where you expected a half-word and every subsequent transfer is offset by one. The data looks like plausible noise, which is the worst kind of wrong.

6 Deep dive 5: Sending and receiving, and a compiler bug

Listing 1: Book Listing 8.4. Polled exchange of one byte. Read the cast carefully.

```

1 uint8_t SPI_Send_Receive_Byte(uint8_t d_out) {
2     uint8_t d_in;
3
4     while ((SPI1->SR & SPI_SR_TXE) == 0)        /* wait for room */
5         ;
6
7     *((uint8_t *)&(SPI1->DR)) = d_out;          /* NOT SPI1->DR = d_out; */
8
9     while ((SPI1->SR & SPI_SR_RXNE) == 0)        /* wait for the exchange */
10        ;
11
12    d_in = (uint8_t) SPI1->DR;
13    return d_in;
14 }
```

The structure is straightforward: wait for TXE, write the data register (which starts the simultaneous transmit and receive, and clears TXE), wait for RXNE, read the data register (which clears RXNE).

But look at that write.

Why `SPI1->DR = d_out;` is wrong

CMSIS declares DR as a 32-bit register, so the compiler generates a **store half-word** (STRH) or store word instruction.

The SPI hardware supports **packing mode**: when the data frame size is 8 bits or less, a **16-bit access to SPIx_DR means you have packed two frames into it**. So the hardware dutifully transmits your byte *and a second, empty one*.

Put a logic analyser on MOSI and you will literally see an extra byte after every correct byte.

The fix is the cast: `*((uint8_t *)&(SPI1->DR)) = d_out;` tells the compiler to emit a **store byte** (STRB) instead, which the hardware reads as one frame.

This is a lovely example of Week 10 mattering in practice. The C is correct in every ordinary sense, and the *instruction the compiler chose* changes the hardware's behaviour. You cannot debug this from the C alone.

6.1 Interrupt-driven

Because writing DR starts a simultaneous exchange, the natural interrupt is RXNE: when the byte exchange completes, the receive buffer fills, the flag sets, and the ISR reads the received data and can load the next byte to transmit. For faster throughput you can instead interrupt on TXE, which sets as soon as the transmit buffer is free, before the exchange has finished.

6.2 The loopback test

Listing 2: Book Listings 8.6–8.7. Wire MOSI to MISO and check.

```

1 void Test_SPI_Loopback(void) {
2     uint8_t out = 'A', in;
3     while (1) {
4         in = SPI_Send_Receive_Byte(out);
5         if (in != out) Control_RGB_LEDs(1, 0, 0); /* red: mismatch */
6         else          Control_RGB_LEDs(0, 1, 0); /* green: match */
7         out++;
8         if (out > 'z') out = 'A';
9     }
10 }
```

Why loopback is the right first test

Physically connect MOSI to MISO so the master receives exactly what it sent.

This is a genuinely good debugging technique because it **removes the slave from the equation**. If loopback works, your clock, your data path, your mode settings, and your code are all fine, and any remaining problem is the slave or its wiring. If loopback fails, stop looking at the sensor.

Recall the CPOL/CPHA trap: since a mode mismatch and a dead wire produce identical symptoms, a test that isolates half the system is worth a great deal.

7 Deep dive 6: Talking to a real sensor

Now the part that generalises. Almost every SPI sensor uses the same register-access convention, and once you know it you can read any of their datasheets.

The command byte

The first byte the master sends encodes what it wants:

Bit 7: 1 for **read**, 0 for **write**.

Bit 6: 1 to enable **auto-increment**, for reading several consecutive registers in one transaction.

Bits [5:0]: the **register address**.

So reading register 0x0F means sending 0x80 | 0x0F = **0x8F**:

$$0x80 = 10000000, \quad 0x0F = 00001111, \quad 0x8F = 10001111$$

Only six bits are left for the address, which is why these sensors have at most 64 registers.

In the lab

Your Lab 08, Task 1, worked end to end.

Powering the sensor first. Before any read, the gyroscope must be turned on by writing CTRL_REG1 at address 0x20. Your code:

```
1 void gyro_init(void) {
2     uint8_t tx[2] = { 0x20, 0x8F };
3     HAL_GPIO_WritePin(GPIOE, GPIO_PIN_3, GPIO_PIN_RESET);    /* CS low
4     /*
5     HAL_SPI_Transmit(&hspi1, tx, 2, HAL_MAX_DELAY);
6     HAL_GPIO_WritePin(GPIOE, GPIO_PIN_3, GPIO_PIN_SET);        /* CS high
7     /*
8 }
```

Note the value 0x8F is 10001111: bit 3 (PD) powers the device up and bits 0–2 enable the X, Y, and Z outputs. The manual's own example writes 0b00001111 instead, which sets the same PD and axis-enable bits without the top bit. Either powers the device; the difference is in the output data rate and bandwidth bits at [7:6], so **check the datasheet's Table for CTRL_REG1 to know which rate you selected**. This is worth resolving, because the ODR determines how fast you can meaningfully sample.

Reading WHO_AM_I. Send 0x8F, then clock in the reply. Expected 0xD3 for the I3G4250D on the blue board. Your project plan also records 0x33 for the LSM303AGR accelerometer's WHO_AM_I, also at register 0x0F, over I²C.

Why WHO_AM_I exists at all, and it is a nice piece of design: it is a read-only register with a known constant value. Recall from Section 3 that SPI has no acknowledgment, so a **fixed-value register is the protocol's substitute for one**. If you read 0xD3, then your clock, mode, chip select, wiring, and command encoding are all correct, because no other combination produces that specific byte by accident. **Always read WHO_AM_I first** when bringing up a new sensor.

7.1 Chip select, in software

Beware the trap

On your board, **PE3 must be driven manually as a GPIO output**, not left to the SPI peripheral's hardware NSS.

Your configuration sets NSS Signal Type to **Software**, meaning the peripheral does not touch the pin and your code is responsible for it. So every transaction is: pull PE3 **low**, do the transfer, pull PE3 **high**.

Forget the low and the sensor never listens. Forget the high and it never deselects, so the next transaction is treated as a continuation of the current one and the register addressing goes wrong from then on.

Recall from Section 3 that CPHA= 1 *requires* slave select to go inactive between transfers, and your sensor uses CPHA= 1. So the high is not optional politeness, it is protocol.

7.2 Polling versus interrupt, in HAL

	Polling	Interrupt
Nature	Blocking, waits for completion	Non-blocking, uses callbacks
Transmit	HAL_SPI_Transmit	HAL_SPI_Transmit_IT
Receive	HAL_SPI_Receive	HAL_SPI_Receive_IT
Completion	Return of the function	HAL_SPI_TxCpltCallback / RxCpltCallback
CubeMX	Nothing extra	Enable SPI1 global interrupt in NVIC

Your Lab 08 required interrupt mode for Task 2 and explicitly stated polling would not be accepted, which is the manual making you feel the difference rather than just read about it.

In the lab

Reading all three axes, and the endianness point from Week 06.

Your project plan's IMU code assembles the readings like this:

```
1 | imu.gz_raw = (int16_t)((buf[5] << 8) | buf[4]);
```

Note the ordering: buf[4] is the **low** byte and buf[5] is the **high** byte, so the sensor sent **low byte first**. Recall Week 06: the STM32 is little-endian, the sensor's byte order is its own business, and you must assemble the value explicitly rather than casting a byte pair. Your code does it correctly.

Note also the **auto-increment bit**. Reading six bytes for three axes in one transaction requires setting bit 6 of the command byte as well as bit 7, so the command for a six-byte burst starting at the first output register is 0xC0 | address. Without it, every read returns the same register six times, which looks like a sensor stuck on one value.

And the third mistake in your own list: **"garbage values on all axes: forgot -u _printf_float in CMakeLists.txt."** That one is not SPI at all. It is the linker omitting floating-point support from printf, so your correctly-read values print as nonsense. Worth remembering that a bad number on screen is not always a bad number in memory.

8 Comparing SPI against UART

Recall Week 14's six problems. Here is where the two protocols land, and next week I will add a third column.

Problem	UART	SPI
Symbol timing	Asynchronous, matched clocks	Synchronous , shared SCK
Framing	Start and stop bits	Slave select assertion
Error detection	Optional parity	None
Acknowledgment	None	None
Media access	Point-to-point only	Master/slave
Addressing	None	One select line per slave
Wires (1 device)	2 (TX, RX)	4 (SCK, MOSI, MISO, CS)
Wires (n devices)	not possible	$3 + n$
Duplex	Independent both ways	Simultaneous exchange
Speed	Slow, MHz at best	Fast, tens of MHz
Settings to agree	Baud, bits, parity, stop, order	CPOL, CPHA, bits, order, speed

The trade, in one line

SPI spends **wires** to buy **speed** and **simplicity**. It has no addressing, no acknowledgment, and no error detection, because the master controls the clock and there is only ever one selected listener, so most of what UART's framing was for simply does not arise.

The price is the select line per slave, and the absence of any way to tell that something went wrong.

9 Tying it back

We asked what changes when the transmitter sends its clock along with the data, and the answer runs in both directions.

You gain **reliability**, because there is no clock mismatch to worry about and therefore no baud-rate tolerance, no start bits, and no framing errors. You gain **speed**, because the receiver never has to guess where a bit boundary is, so megahertz rates are ordinary rather than exotic. And you gain **simultaneous exchange**, because master and slave form one circular shift register, which is why reading a register means clocking out a dummy byte to clock in the answer.

You lose the two-wire simplicity, since every additional slave needs another select line, and you lose every form of feedback: no parity, no acknowledgment, no framing error. So a wrong **CPOL** or **CPHA**, an unasserted chip select, and a disconnected wire all produce exactly the same symptom, which is why a known-value register like `WHO_AM_I` is the standard first test and a logic analyser is close to essential.

And along the way, two bugs worth having met: the **packing-mode phantom byte** caused by the compiler choosing `STRH` over `STRB`, and the `FRXTH` **threshold** that must match your data size or every subsequent transfer is offset by a byte.

Next week, I²C, which takes SPI's synchronous clock and asks whether the select lines can be eliminated by putting addresses back into the message. The answer is yes, and it costs you speed and a great deal of protocol complexity.

Cheat sheet: Week 15

The four signals

SCK clock, **generated by the master only** · MOSI master out · MISO master in · NSS/CS **one line per slave**.

SPI has no addressing in the protocol. The master asserts one select line; everyone else ignores the bus. So n slaves need n select pins, and $3 + n$ wires total.

Drop MISO or MOSI if communication is one-way.

Simultaneous exchange

Master and slave shift registers form a **circular loop**. **Every clock pulse exchanges one bit**. Eight pulses = one byte out *and* one byte in.

There is no transmit-only or receive-only. Every transaction is an exchange.

Therefore reading a register takes two byte-times: send the command (reply is meaningless), then clock a **dummy byte** to shift the answer out. *The clock only runs while the master transmits.*

CPOL and CPHA — the exam question and the commonest bug

CPOL: 0 = active-high, idles low. 1 = active-low, idles high.

CPHA: 0 = data valid from slave-select assertion, sampled on the **first** edge; select *may stay active* between transfers. 1 = data presented on the first edge, sampled on the **second**; select **must go inactive** between transfers.

Mode 0 (0,0) · **Mode 1** (0,1) · **Mode 2** (1,0) · **Mode 3** (1,1).

There is no error flag for a mode mismatch. Wrong mode, unasserted CS, dead wire, and unpowered sensor **all look identical**. Read the mode from the slave's datasheet; it is not discoverable.

Your gyroscope: CPOL High, CPHA 2nd Edge = mode 3.

The peripheral

CR1: MSTR · LSBFIRST · CPOL · CPHA · BR · SPE.

SPE disables further configuration changes — configure first, enable last (like the timer's CEN).

CR2: DS data size (4–16 bits, default 8) · TXEIE/RXNEIE/ERRIE · FRXTH.

SR: TXE · RXNE · FTLVL/FRLVL fill levels · BSY · MODF mode fault (two masters).

Two 4-byte FIFOs ease software timing.

FRXTH must match data size: 0 for >8-bit data, 1 otherwise. Wrong value offsets every transfer by a byte.

$$\text{Baud} = \frac{f_{\text{BusClock}}}{2^{\text{BR}+1}}$$

Book: $\text{BR} = 3 \Rightarrow 48/16 = 3 \text{ MHz}$. **Yours:** prescaler 8 ($\text{BR} = 2$) $\Rightarrow 48/8 = 6 \text{ MHz}$, under the sensor's 10 MHz limit. *The ceiling comes from the slave.*

The packing-mode bug

$\text{SPI1->DR} = \text{d_out}$; makes the compiler emit STRH (16-bit). With $\text{DS} \leq 8$ bits, a 16-bit access means **two packed frames**, so the hardware **transmits a phantom extra byte**.

Fix: `*((uint8_t *)&(SPI1->DR)) = d_out;` forces STRB.

Visible on a logic analyser as an extra byte after every correct one. Week 10 mattering in practice.

Loopback test: wire MOSI to MISO. Removes the slave from the equation — if it works, clock/data/mode/code are fine.

Talking to a sensor

Command byte: bit 7 = 1 read / 0 write · bit 6 = auto-increment · bits [5:0] = register address.

Read 0x0F ⇒ send 0x80 | 0x0F = 0x8F. Six address bits ⇒ at most 64 registers.

Burst read of 6 bytes ⇒ set bit 6 too: 0xC0 | address. **Without it you read the same register repeatedly.**

WHO_AM_I is the protocol's substitute for an acknowledgment — a read-only register with a known constant. **Always read it first.**

I3G4250D ⇒ 0xD3. LSM303AGR ⇒ 0x33. Both at register 0x0F.

Power the gyro first by writing CTRL_REG1 (0x20): bit 3 PD, bits 0–2 axis enables.

Your CS is PE3, software-controlled: low before, high after, every transaction. CPHA= 1 **requires** the deassertion.

Sensor sends **low byte first:** `(int16_t)((buf[5] << 8) | buf[4])`. Never cast a byte pair.

SPI vs UART

SPI: **synchronous**, framing by chip select, **no error detection, no acknowledgment**, master/slave, **one select line per slave, simultaneous exchange**, fast (tens of MHz).

UART: asynchronous, start/stop framing, optional parity, no acknowledgment, point-to-point only, independent TX and RX, slow.

SPI spends wires to buy speed and simplicity, and gives up all feedback in return.

Likely exam prompts from this section

- Name the four SPI signals and their directions. *Near certain.*
- Explain CPOL and CPHA and enumerate the four modes.
- Why does SPI need no address field, and what does that cost?
- Why must you transmit a dummy byte to receive data?
- Compute the SPI clock from the bus clock and BR.
- Compare SPI and UART across the six problems from Week 14.
- Why is reading WHO_AM_I the standard first step with a new sensor?